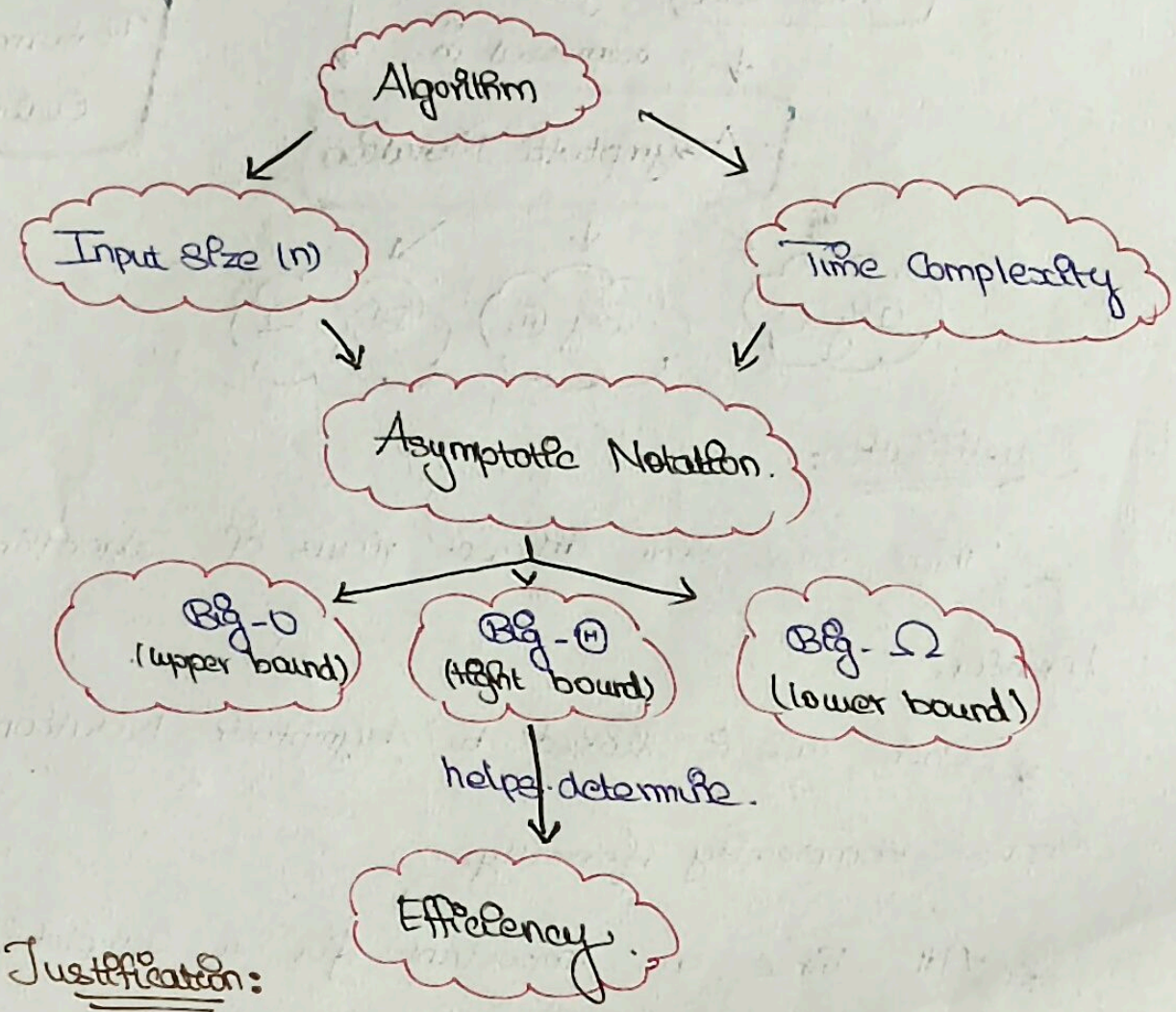
Name: Vegashree G

Reg No: 192573022

Course: CSA0620 Design and Analysis of Algorithms.

CO1-AT2 CONCEPT MAPPING.

(Q1) **Algorithm Efficiency and Asymptotic Notation:**



Justification:

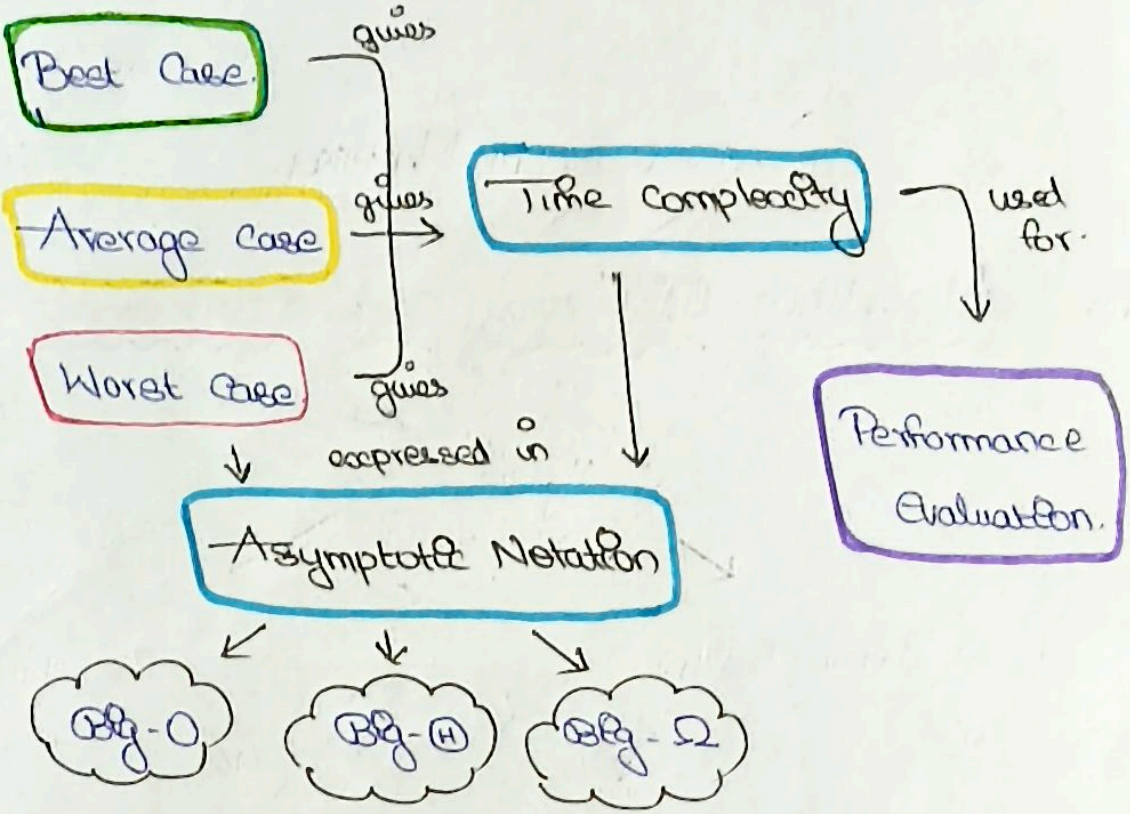
↳ As Input size (n) affects the time taken by the algorithm

↳ Time complexity shows how the time grows with n.

↳ Asymptotic notations classifies the growth for large n.

(Big-O, Big-Θ, Big-Ω)

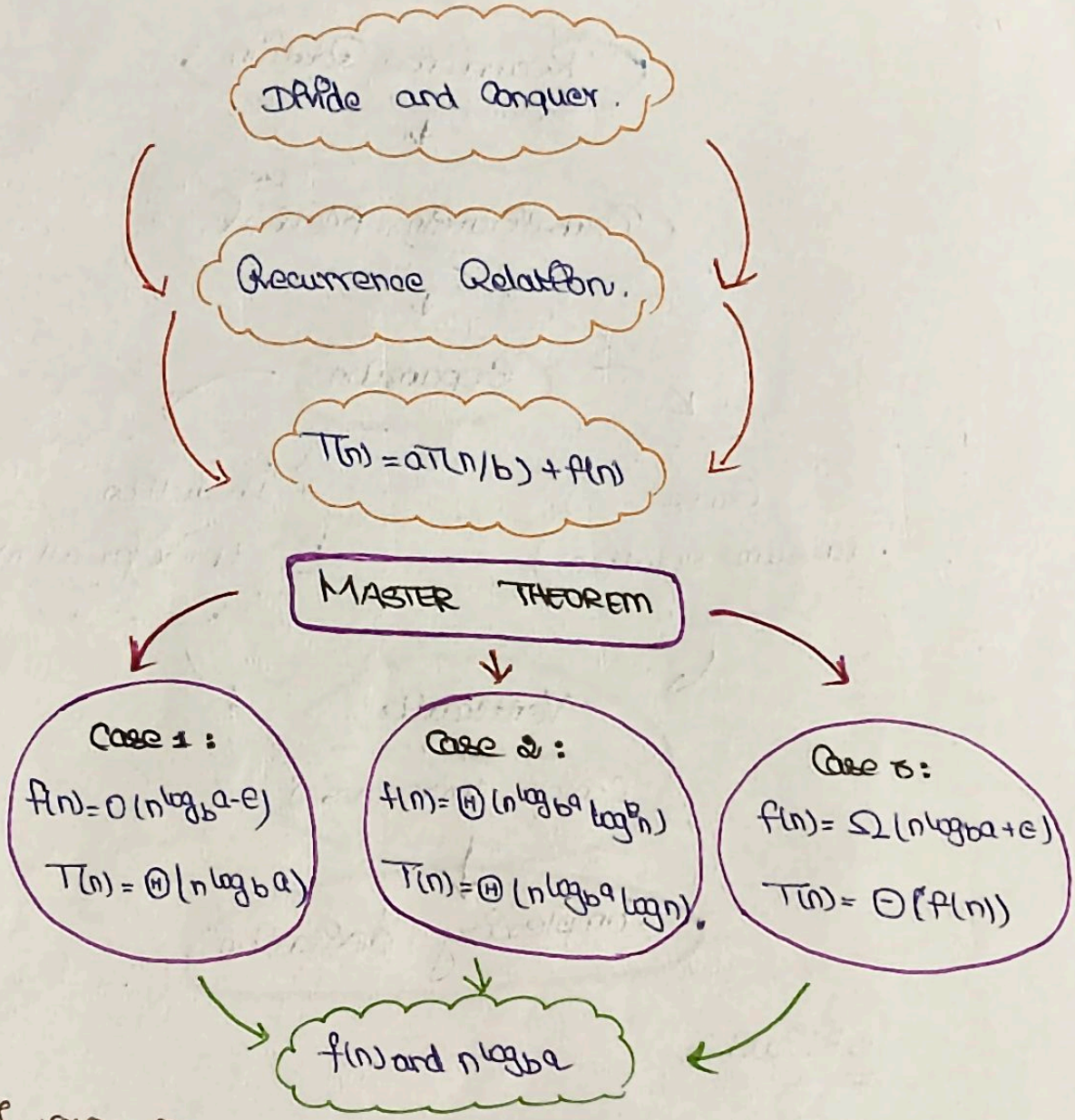
Q10) Best-Case, Average-Case and Worst Case Analysis.



Justification:

* Three cases show different views of algorithm behavior.
* Each case is linked to Asymptotic Notation to express complexity formally.
* All three are important for a complete performance analysis.
* Helps to compare practical behavior Vs theoretical behavior.

Q3) Divide and Conquer - Master Theorem:



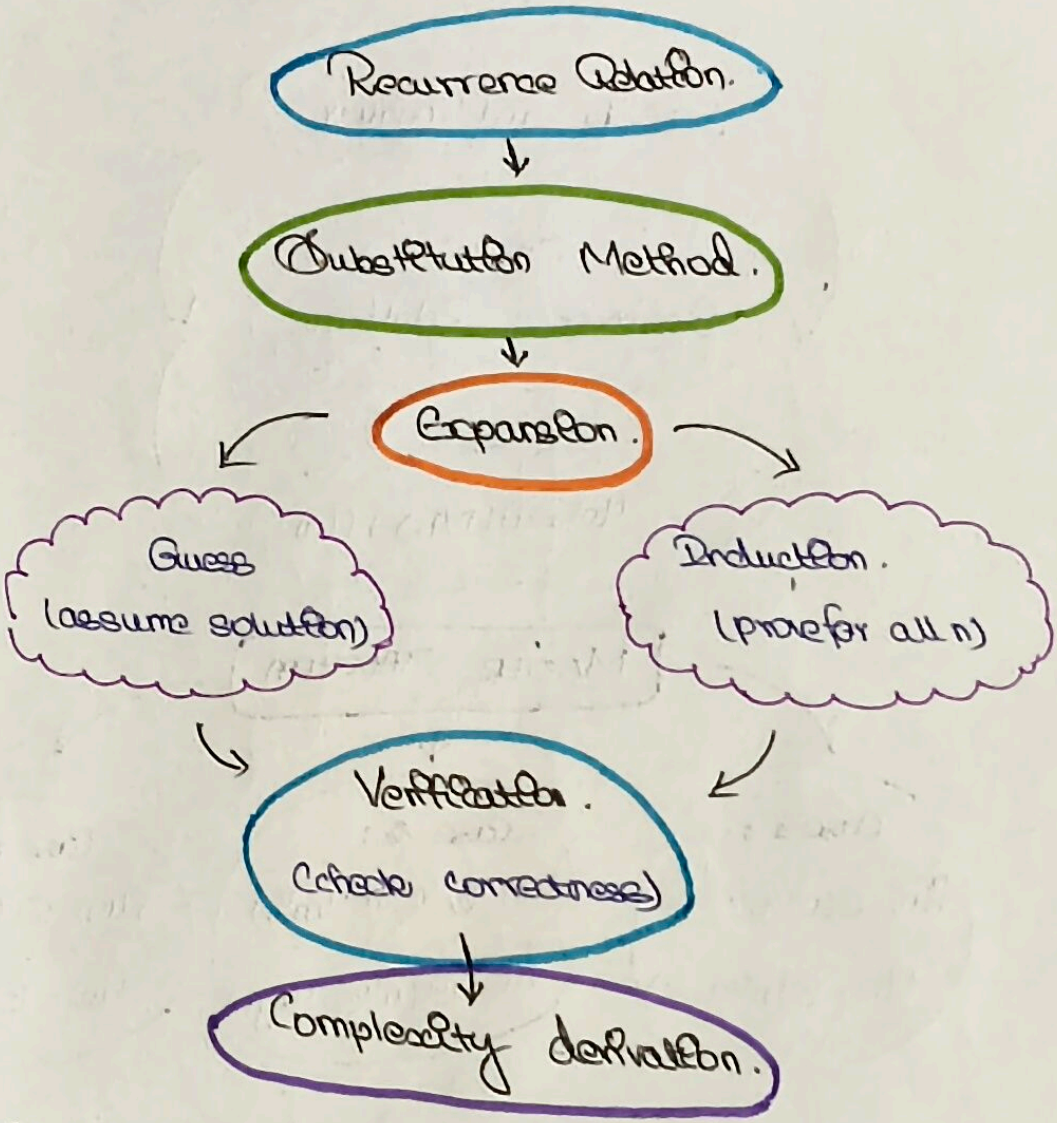
Justification:

↳ Each nodes leads to the next in analysing recursive algorithms.

↳ The three cases depends on the comparison of f(n) and $n^{\log_b a}$. Simplifies deriving time complexity.

↳ Helps analyze divide-and-conquer algorithms efficiently.

Q4) Substitution Method for Solving Recurrence Relations



Justification:

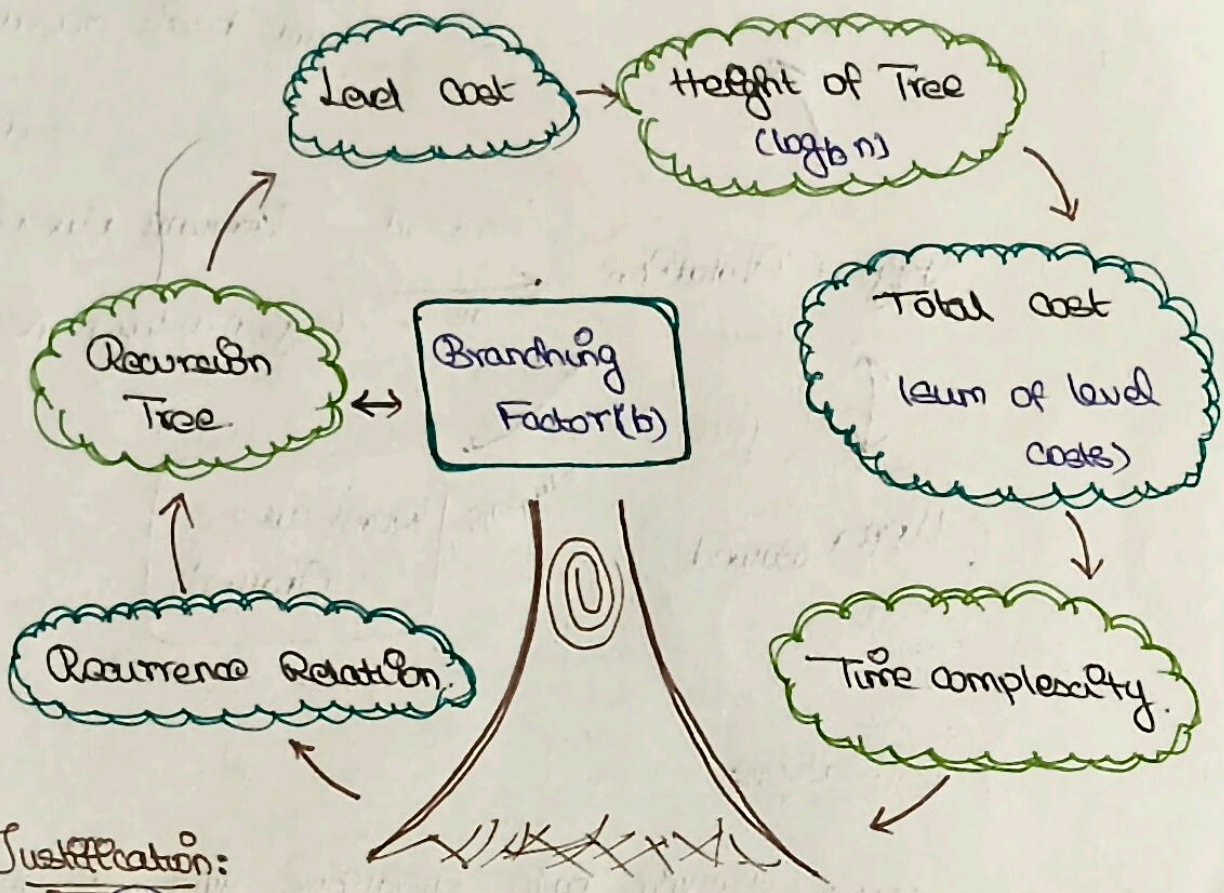
→ Substitute the guessed form into the recurrence.

→ Guess + Induction support the proof.

→ Verification confirms the correctness of the solution. Provides a stepwise framework for solving recurrences.

→ Use induction to prove it.

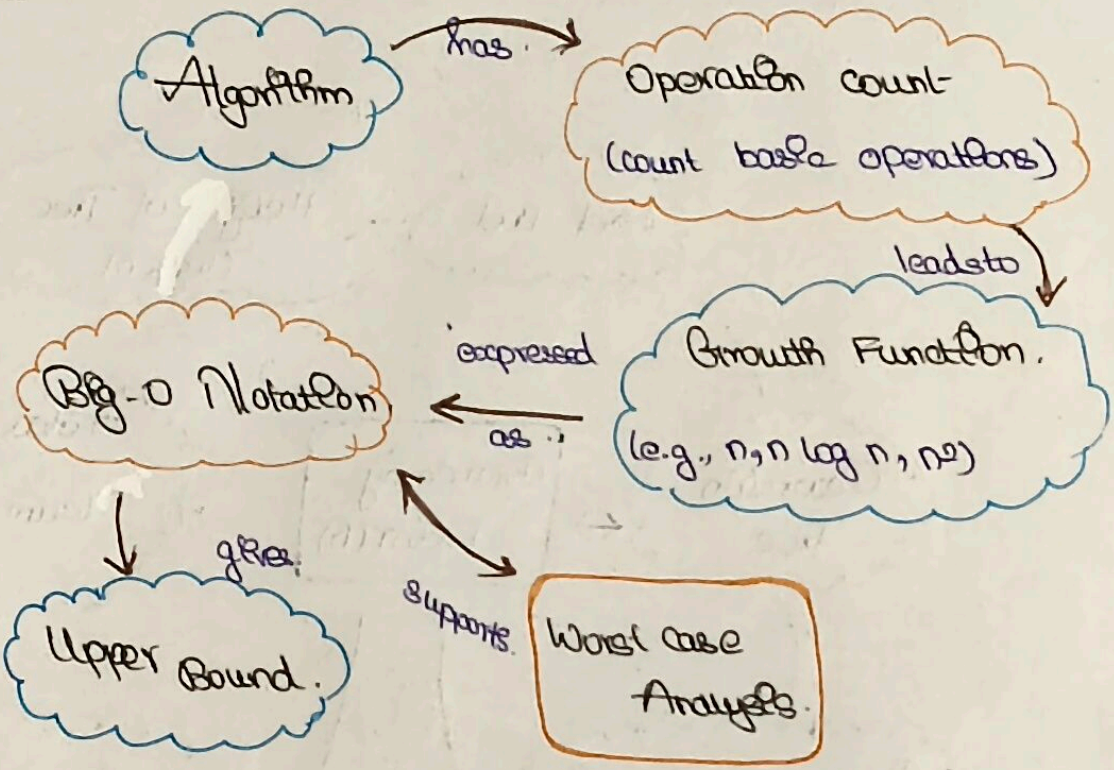
Q5) Recursion Tree Methods for Solving Recurrence Relations.



Justification:

* → Recursion tree representing the recurrence vis ually. Cost is distributed across level.
* → Branching factor supports the tree structure.
* → Summing level-wise costs gives total cost.
* → Makes recursive analysis visual and easier to understand.

Q6) (v) Operation Count and Big-O Notation:



Justification:

→ Counting basic operations gives a growth function.

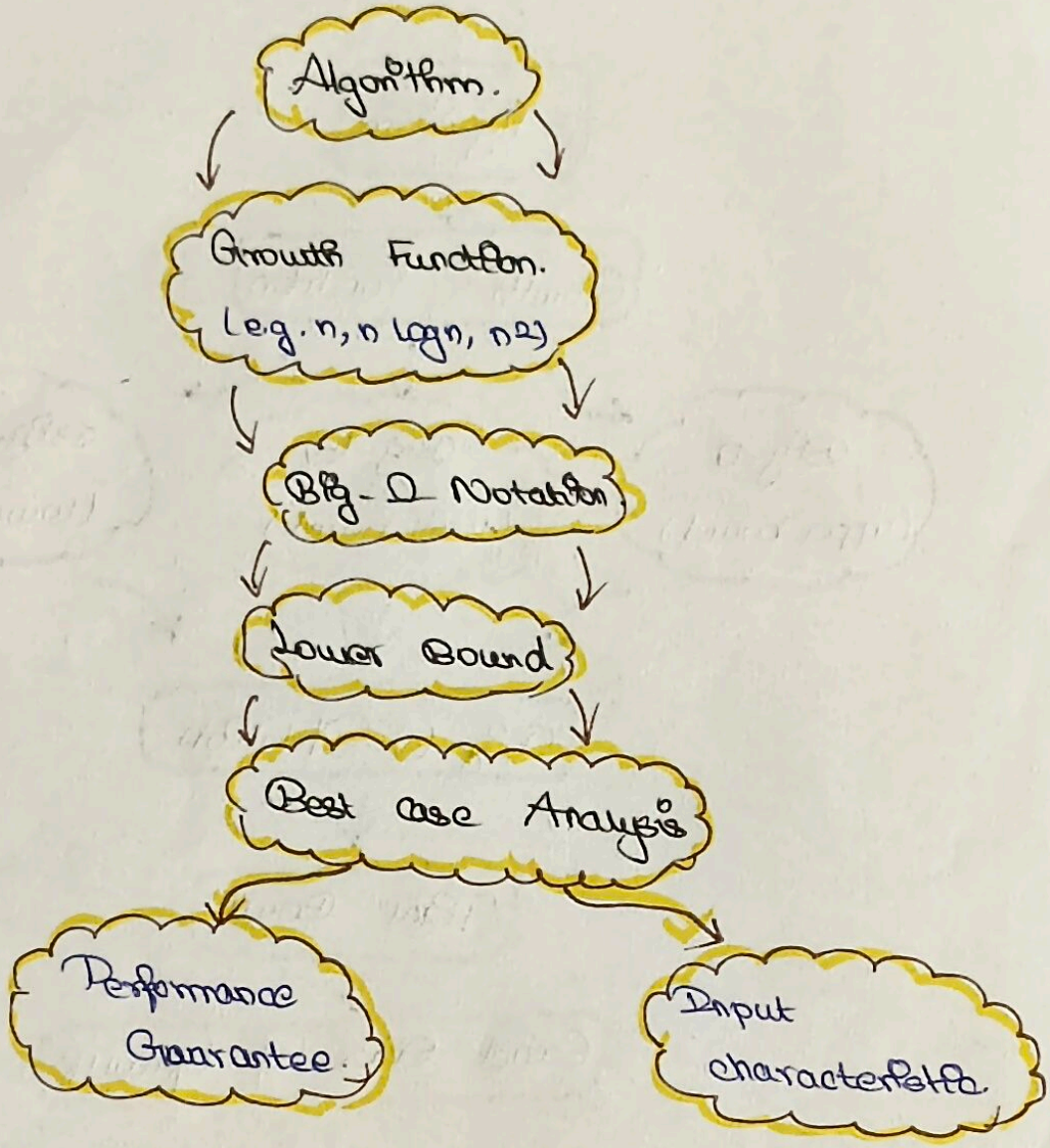
→ Big-O provides an upper bound on running time.

→ Worst case analysis supports Big-O.

→ Used for reliable performance estimation for large inputs.

Big-Omega Notation and Lower Bound Analysis:

Q#1)



Justification:

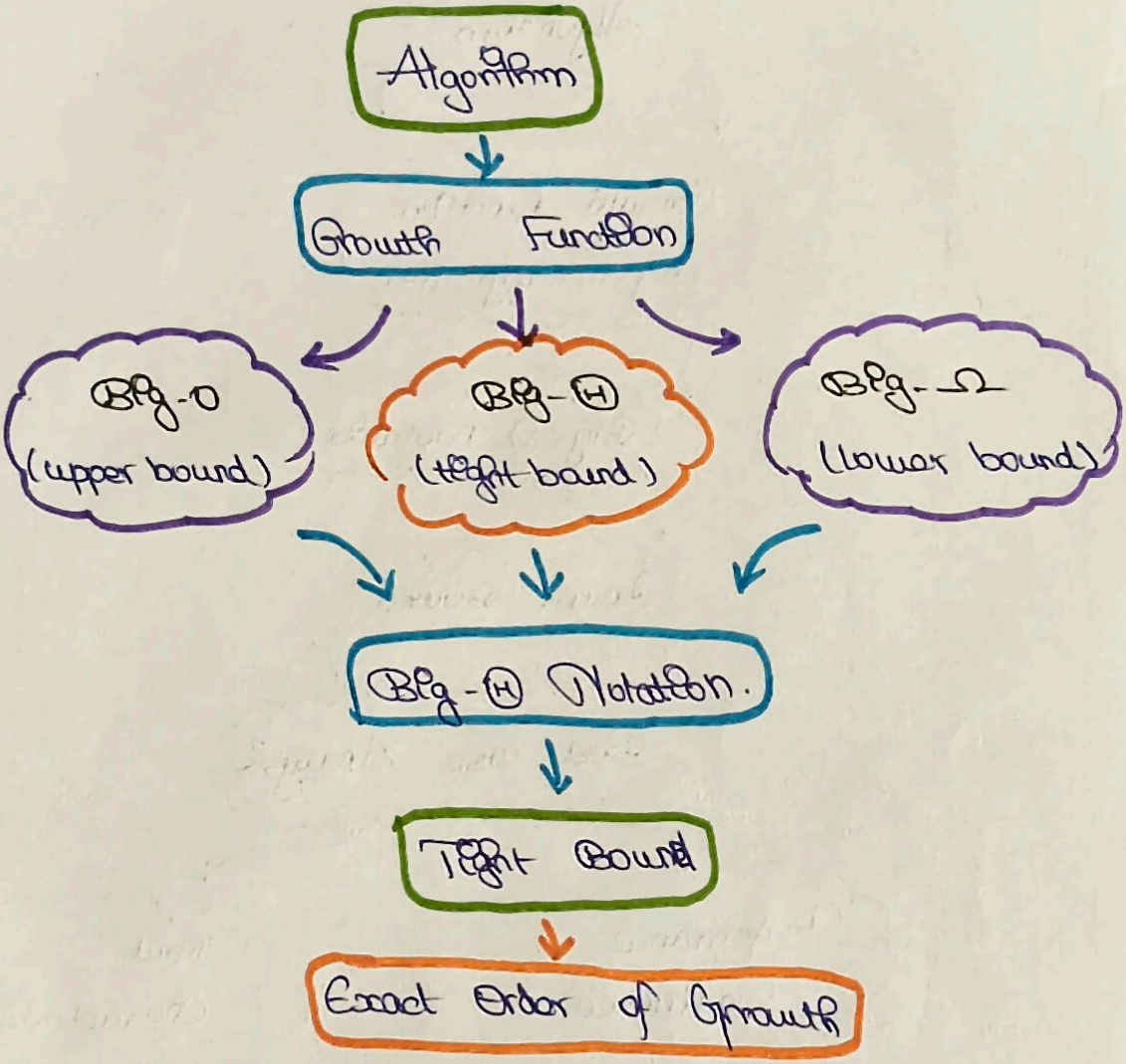
★ Big-Ω shows the minimum behaviour.

★ Lower bound comes from Big-Ω.

★ Performance gurantee and input characteristic support analysis.

★ Gives a balanced view from the lower-bound perspective.

(Q8) Big-Theta Notation and Tight Bound :



Justification:

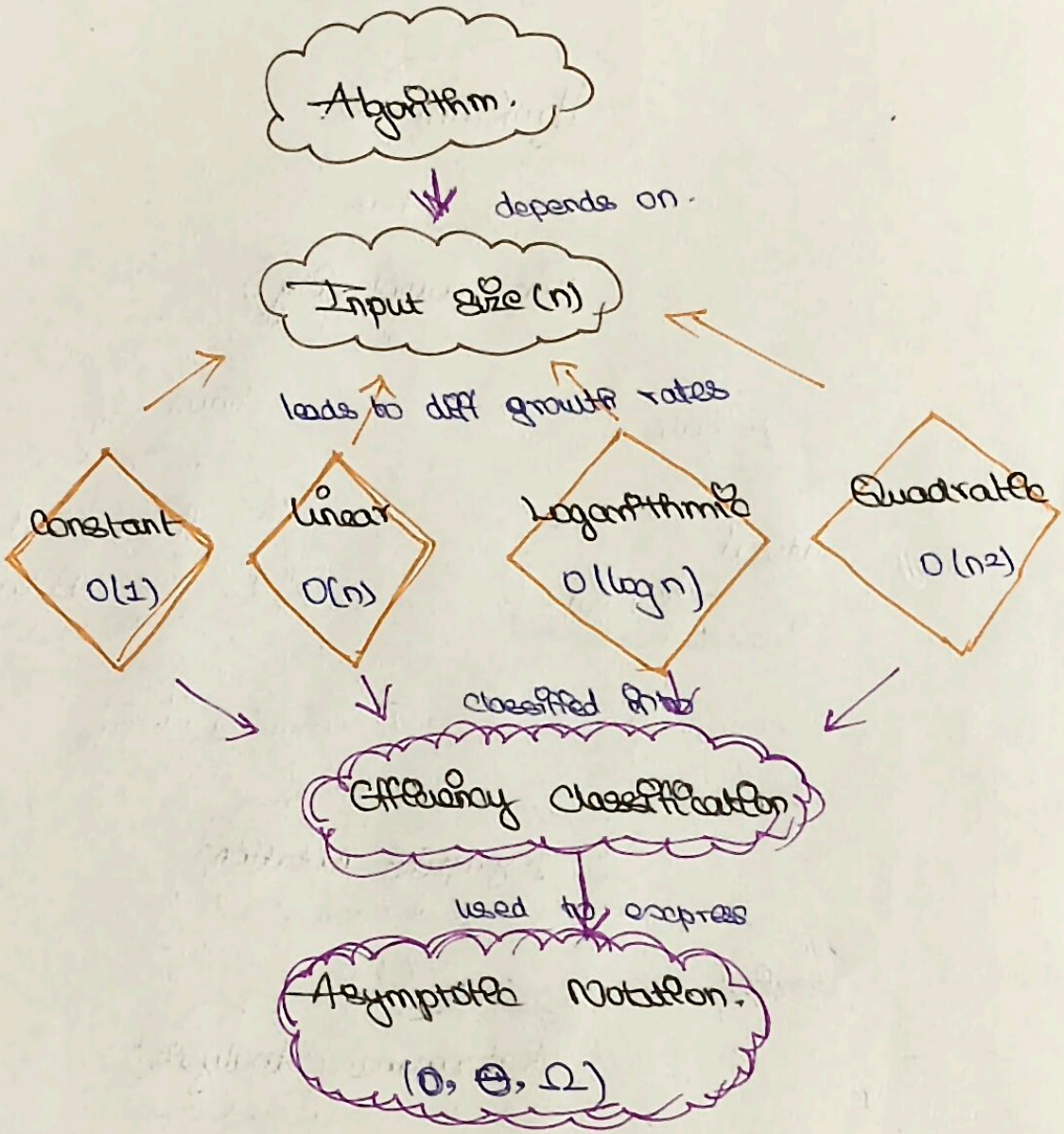
→ Tight bound occurs when upper and lower bounds meet. Big O and Big Ω both leads to Big-Θ

→ Big-Θ is preferred when exact growth is known

→ Helps distinguish approximate vs precise complexity

Q9)

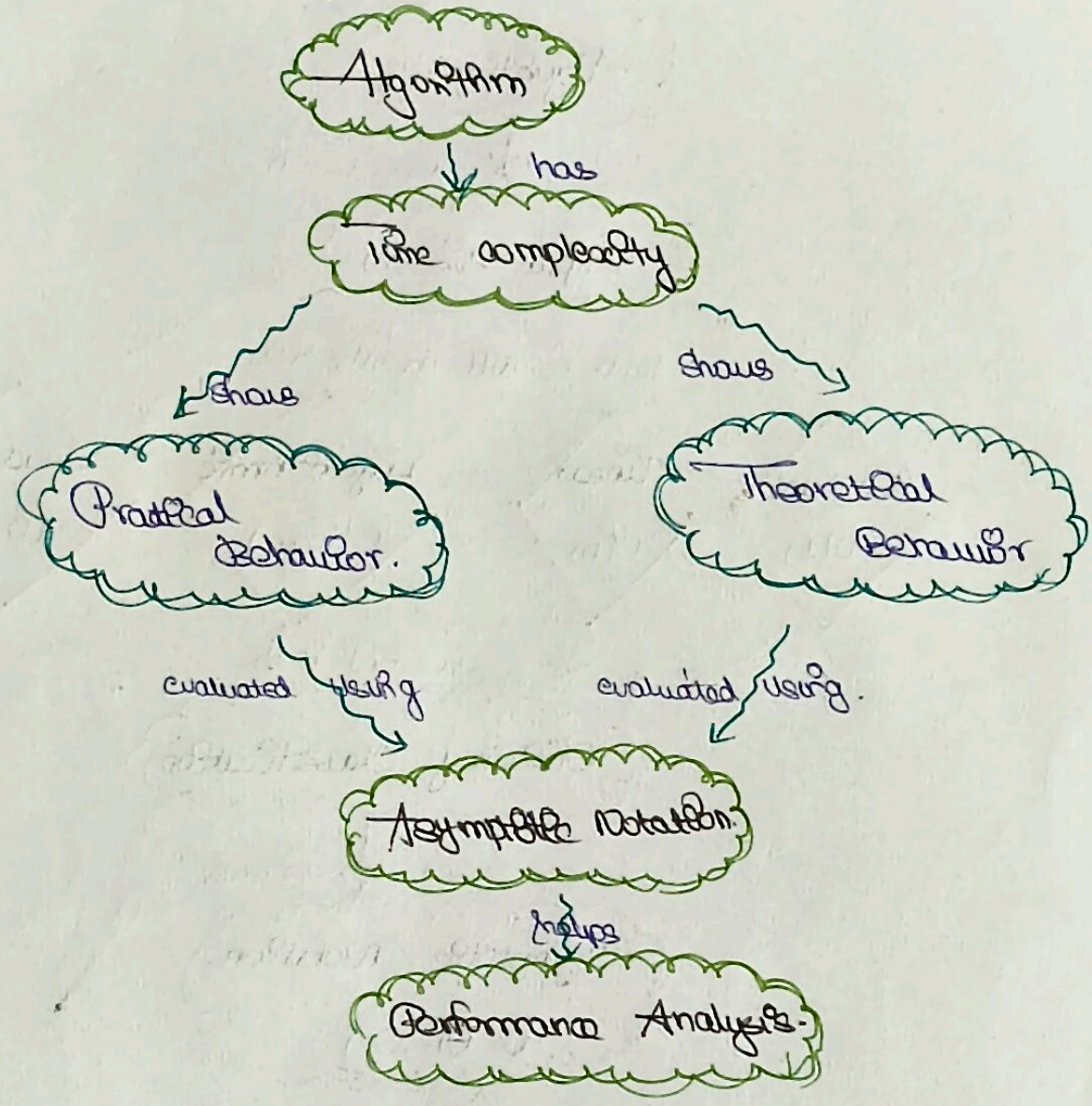
Time Complexity Classification:



Justification:

- Larger input sizes show different growth patterns.
- Each pattern is classified (constant, linear, logarithmic, quadratic).
- Asymptotic Notation is linked to each class.
- Log and linear are more scalable than quadratic.

Q10) Comparison of Linear and Binary Search:



Justification:

- Shows how upper and lower bounds combine for precise analysis.
- Provides a complete understanding of algorithm performance.
- Useful for real-world and academic analysis.